

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA14

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO5: *Develop code optimization and Code Generation using DAG representation with data flow analysis to produce optimized target code. (BL3)*

Assessment Tool Weight-age: 40%

QUESTIONS: 5

Total Marks:50

Annexure A

SIMULATION TASK

Code of Conduct:

I N.SANDIYANITTIANANDAM(192571029) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Criteria	Weight age (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	20%	Demonstrates strong understanding of underlying concepts in the simulation	Shows good understanding with minor gaps	Partial understanding; some misconceptions	Lacks understanding of key concepts
Model Design	25%	Develops accurate, well-structured simulation/model independently	Develops correct model with minor errors	Model is incomplete or requires guidance	Unable to develop a proper model

Tool Usage	20%	Uses simulation tools effectively with correct setup and execution	Uses tools with minor errors or guidance	Limited ability to use tools properly	Unable to use simulation tools
Analysis of Results	20%	Provides clear, logical analysis and meaningful interpretation of results	Analysis is reasonable with minor gaps	Superficial analysis; limited interpretation	Unable to analyze or interpret results
Documentation	15%	Presents results clearly with proper documentation and structured reporting	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation

Annexure A

SIMULATION TASK

QUESTIONS

1. Given the three-address code sequence:

```
t1 = x * 1
t2 = t1 + y
t3 = t2 - 0
t4 = t3 * 0
if t4 != 0 goto L1
t5 = y + y
```

Simulate the **peephole optimization** process on this sequence. Produce the optimized instruction sequence and justify each optimization step applied, including algebraic simplification, constant folding, and dead code elimination.

SOLUTION

Peephole Optimization

Given the three-address code:

```
t1 = x * 1
t2 = t1 + y
t3 = t2 - 0
t4 = t3 * 0
if t4 != 0 goto L1
t5 = y + y
```

1. Algebraic Simplification

Apply basic algebraic identities:

- $x * 1 = x$
- $a - 0 = a$
- $a * 0 = 0$

Therefore:

```
t1 = x
t2 = t1 + y
t3 = t2
t4 = 0
if t4 != 0 goto L1
t5 = y + y
```

We can further substitute temporary values:

```
t1 = x
t2 = x + y
t3 = t2
t4 = 0
if t4 != 0 goto L1
t5 = y + y
```

2. Constant Folding

The condition becomes:

```
if 0 != 0 goto L1
```

Since $0 \neq 0$ is always **false**, the conditional jump can never be taken.

Thus:

```
goto L1
```

is **not required**. The conditional branch can be removed.

3. Dead Code Elimination

Because the condition is always false, L1 will never be reached through this branch.

Also, t1, t2, t3, and t4 are only used to calculate this always-false condition. Therefore, their computations are unnecessary and can be eliminated.

The instruction:

```
t5 = y + y
```

is retained because it occurs after the condition and may produce a value or side effect depending on the surrounding program.

4. Optimized Instruction Sequence

The optimized sequence is:

```
t5 = y + y
```

Optimization Summary

Original Instruction	Optimization
t1 = x * 1	$x * 1 = x$
t2 = t1 + y	Simplified/substituted to $x + y$
t3 = t2 - 0	$a - 0 = a$
t4 = t3 * 0	$a * 0 = 0$
if t4 != 0 goto L1	if $0 \neq 0$ is always false
t5 = y + y	Retained

2.Partition the following intermediate code into **basic blocks** and construct the corresponding **control flow graph**. Identify all leader statements and mention the rules used to identify them:

```
(1) x = a + b
(2) y = x * c
(3) if y > 50 goto 6
(4) z = y - d
(5) goto 7
(6) z = y + d
(7) w = z * 2
```

SOLUTION

Basic Blocks and Control Flow Graph

Given the intermediate code:

```
(1) x = a + b
(2) y = x * c
(3) if y > 50 goto 6
(4) z = y - d
(5) goto 7
(6) z = y + d
(7) w = z * 2
```

1. Rules for Identifying Leaders

A statement is a leader if:

1. The first statement of the intermediate code is a leader.
2. Any statement that is the target of a conditional or unconditional jump is a leader.
3. Any statement immediately following a jump statement is a leader.

2. Identify the Leaders

- Statement (1) → Leader because it is the first statement.
- Statement (4) → Leader because it immediately follows the conditional jump at (3).
- Statement (6) → Leader because it is the target of goto 6.
- Statement (7) → Leader because it is the target of goto 7.

Therefore:

Leaders={1,4,6,7}

3. Construct Basic Blocks

Basic Block B1

Statements (1)–(3):

B1:

(1) $x = a + b$
 (2) $y = x * c$
 (3) if $y > 50$ goto 6

Basic Block B2

Statements (4)–(5):

B2:
 (4) $z = y - d$
 (5) goto 7

Basic Block B3

Statement (6):

B3:
 (6) $z = y + d$

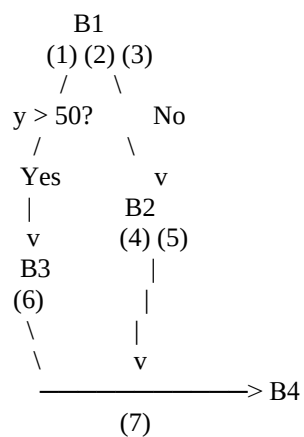
Basic Block B4

Statement (7):

B4:
 (7) $w = z * 2$

4. Control Flow Graph

The control flow is:



More precisely, the edges are:

- $B1 \rightarrow B3$ when $y > 50$ is true.
- $B1 \rightarrow B2$ when $y > 50$ is false.
- $B2 \rightarrow B4$ because of goto 7.
- $B3 \rightarrow B4$ because execution continues from statement (6) to (7).

5. Final Answer

Basic Block	Statements
B1	(1), (2), (3)
B2	(4), (5)
B3	(6)
B4	(7)

Leaders: 1, 4, 6, 7

The CFG represents the two possible paths after the condition $y > 50$: either execute $z = y - d$ or $z = y + d$, after which both paths merge at statement (7).

3. Construct the **DAG** for the following basic block:

$p = a + b$
 $q = a + b$
 $r = p * q$
 $s = a + b$
 $t = r + s$

Using the DAG, identify all **common sub-expressions**. Then simulate a simple code generator to produce optimized target code and explain the register allocation decisions at each step.

SOLUTION

DAG Construction and Code Generation

Given the basic block:

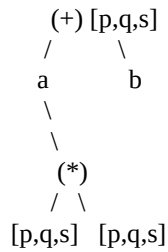
$p = a + b$
 $q = a + b$
 $r = p * q$
 $s = a + b$
 $t = r + s$

1. DAG Construction

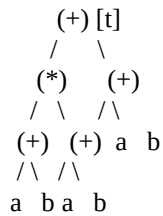
The expression $a + b$ occurs three times:

$p = a + b$
 $q = a + b$
 $s = a + b$

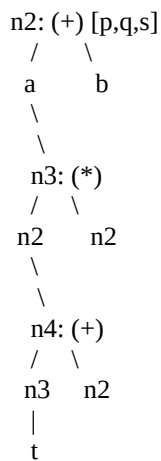
In a DAG, the same expression is represented by one common node and has three labels: p , q , and s .



More clearly, the DAG structure is:



Since p, q, and s all represent the same $a+b$ node, the optimized DAG can be represented as:



Thus:

- $n2 = a + b \rightarrow \text{labels } \{p, q, s\}$
- $n3 = n2 * n2 \rightarrow \text{label } \{r\}$
- $n4 = n3 + n2 \rightarrow \text{label } \{t\}$

2. Common Sub-expressions

The common sub-expression is:

$a+b$

It appears three times:

$p = a + b$
 $q = a + b$
 $s = a + b$

Therefore, instead of calculating $a+b$ three times, the compiler calculates it once and reuses the result.

3. Optimized Intermediate Code

The original code:

```
p = a + b
q = a + b
r = p * q
s = a + b
t = r + s
```

can be optimized to:

```
p = a + b
r = p * p
t = r + p
```

Here, q and s do not require separate calculations because they have the same value as p .

4. Simple Code Generation

Assume two registers: R1 and R2.

Step 1: Calculate $a + b$

```
MOV R1, a
ADD R1, b
```

Now:

$R1 = a + b$

Since p , q , and s all have this value, R1 is reused.

Step 2: Calculate $r = p * q$

Since both p and q equal R1:

```
MOV R2, R1
MUL R2, R1
```

Now:

$R2 = (a + b) * (a + b)$

This is r .

Step 3: Calculate $t = r + s$

Since $s = a + b$ and that value is still available in R1:

ADD R2, R1

Now:

$R2 = (a + b)^2 + (a + b)$

Therefore:

MOV t, R2

5. Optimized Target Code

```
MOV R1, a
ADD R1, b
MOV R2, R1
MUL R2, R1
ADD R2, R1
MOV t, R2
```

6. Register Allocation Decisions

Step	Operation	Register	Reason
1	$a + b$	R1	Store common sub-expression
2	Copy p to R2	R2	Preserve $a+b$ in R1
3	$R2 * R1$	R2	Compute $r = p*q$
4	$R2 + R1$	R2	Add $s = a+b$
5	Store result	t	Final result

4. For a target machine with two registers **R0** and **R1**, simulate the working of a **simple code generator** for the code:

```
t1 = a + b
t2 = c * d
t3 = t1 - t2
t4 = e + f
t5 = t3 / t4
```

Show the **register descriptor** and **address descriptor** after each instruction is processed.

SOLUTION

Simple Code Generator with Register and Address Descriptors

Given:

t1 = a + b
t2 = c * d
t3 = t1 - t2
t4 = e + f
t5 = t3 / t4

Target machine has **two registers: R0 and R1.**

1. Initial Descriptors

Initially, all variables are stored in memory.

Register Descriptor

Register Contents

R0 —
R1 —

Address Descriptor

Variable	Location
a	Memory
b	Memory
c	Memory
d	Memory
e	Memory
f	Memory
t1–t5	Not generated

2. Process t1 = a + b

Load a into R0 and add b.

MOV R0, a
ADD R0, b

Now:

R0 = t1 = a + b
R1 = empty

Register Descriptor

Register Contents

R0	t1
R1	—

Address Descriptor**Variable Location**

a	Memory
b	Memory
t1	R0

3. Process $t2 = c * d$

Use R1 for t2.

```
MOV R1, c
MUL R1, d
```

Now:

```
R0 = t1
R1 = t2
```

Register Descriptor**Register Contents**

R0	t1
R1	t2

Address Descriptor**Variable Location**

t1	R0
t2	R1
a, b, c, d	Memory

4. Process $t3 = t1 - t2$

Both operands are already in registers, so no additional loads are needed.

```
SUB R0, R1
```

Now R0 contains t3.

Register Descriptor

Register Contents

R0	t3
R1	t2

Address Descriptor

Variable Location

t1	R0*
t2	R1
t3	R0

* means the old value of t1 has been replaced by t3.

5. Process $t4 = e + f$

t3 in R0 is still needed for the final division, so we must preserve it. R1 can be reused after its old value t2 is no longer needed.

```
MOV R1, e
ADD R1, f
```

Now:

```
R0 = t3
R1 = t4
```

Register Descriptor

Register Contents

R0	t3
R1	t4

Address Descriptor

Variable Location

t3	R0
t4	R1
e, f	Memory

6. Process $t5 = t3 / t4$

Both operands are available in registers.

DIV R0, R1

Now R0 contains the final result t5.

Register Descriptor

Register Contents

R0 t5

R1 t4

Address Descriptor

Variable Location

t3 R0*

t4 R1

t5 R0

7. Complete Target Code

MOV R0, a

ADD R0, b ; t1

MOV R1, c

MUL R1, d ; t2

SUB R0, R1 ; t3

MOV R1, e

ADD R1, f ; t4

DIV R0, R1 ; t5

8. Descriptor Summary

Instruction	R0	R1	Important Address Descriptor
Initial	—	—	All variables → Memory
t1 = a+b	t1	—	t1 → R0
t2 = c*d	t1	t2	t1 → R0, t2 → R1
t3 = t1-t2	t3	t2	t3 → R0, t2 → R1
t4 = e+f	t3	t4	t3 → R0, t4 → R1
t5 = t3/t4	t5	t4	t5 → R0, t4 → R1

5. For a **RISC-style target machine** with instructions **LOAD, STORE, ADD, SUB, MUL, MOV**, simulate target code generation for the expression:

$(a - b) * (c + d) - e$

Show the complete sequence of target instructions generated. Also explain the **runtime storage management** and **stack frame layout** used during the computation.

Given the three-address code sequence:

```
t1 = a + b,  
t2 = t1 + 0,  
t3 = t2 * 1,  
if t3 == 0 goto L1,  
t4 = t3 + t3
```

SOLUTION

RISC-Style Target Code Generation

Given the expression:

$(a-b) \times (c+d) - e$

Assume the variables a, b, c, d, e are stored in memory and registers are available for temporary values.

1. Three-Address Representation

The expression can first be represented as:

```
t1 = a - b  
t2 = c + d  
t3 = t1 * t2  
t4 = t3 - e
```

2. Target Code Generation

A possible RISC-style instruction sequence is:

```
LOAD R0, a  
LOAD R1, b  
SUB  R0, R0, R1    ; R0 = a - b  
  
LOAD R1, c  
LOAD R2, d  
ADD  R1, R1, R2    ; R1 = c + d  
  
MUL  R0, R0, R1    ; R0 = (a-b)*(c+d)  
  
LOAD R1, e  
SUB  R0, R0, R1    ; R0 = (a-b)*(c+d)-e
```

STORE result, R0

3. Step-by-Step Execution

Step	Instruction	Result
1	LOAD R0, a	$R0 = a$
2	LOAD R1, b	$R1 = b$
3	SUB R0,R0,R1	$R0 = a-b$
4	LOAD R1,c	$R1 = c$
5	LOAD R2,d	$R2 = d$
6	ADD R1,R1,R2	$R1 = c+d$
7	MUL R0,R0,R1	$R0 = (a-b)(c+d)$
8	LOAD R1,e	$R1 = e$
9	SUB R0,R0,R1	$R0 = (a-b)(c+d)-e$
10	STORE result,R0	Result stored in memory

4. Runtime Storage Management

During execution, the program needs storage for:

- Input variables: a, b, c, d, e
- Temporary values
- Final result
- Return information and other runtime information, if the expression is inside a function

The **registers** hold frequently used intermediate values, while memory is used for variables and values that cannot remain in registers.

5. Stack Frame Layout

A simple stack frame can be represented as:

Higher Address	
Return address	
Saved registers	
Local variables	
Temporary values t1, t2, t3, t4	
Alignment / padding	
Lower Address	

During computation, temporary values can either remain in registers or be **spilled to the stack** when registers are insufficient.

6. Optimization of the Given Three-Address Code

You also provided:

```
t1 = a + b
t2 = t1 + 0
t3 = t2 * 1
if t3 == 0 goto L1
t4 = t3 + t3
```

Apply algebraic simplification:

- $t1 + 0 = t1$
- $t2 * 1 = t2$

Therefore:

```
t1 = a + b
t2 = t1
t3 = t2
if t3 == 0 goto L1
t4 = t3 + t3
```

These assignments can be eliminated through **copy propagation**:

```
t1 = a + b
if t1 == 0 goto L1
t4 = t1 + t1
```

Optimized Code

```
t1 = a + b
if t1 == 0 goto L1
t4 = t1 + t1
```

This optimization reduces unnecessary additions and multiplications, decreases the number of temporary assignments, and improves execution efficiency.

RUBRICS FOR CALCULATION

Simulation tools

Criteria	Weight age (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	20%	Demonstrates strong understanding of underlying concepts in the simulation	Shows good understanding with minor gaps	Partial understanding; some misconceptions	Lacks understanding of key concepts
Model Design	25%	Develops accurate, well-structured simulation/model independently	Develops correct model with minor errors	Model is incomplete or requires guidance	Unable to develop a proper model
Tool Usage	20%	Uses simulation tools effectively with correct setup and execution	Uses tools with minor errors or guidance	Limited ability to use tools properly	Unable to use simulation tools
Analysis of Results	20%	Provides clear, logical analysis and meaningful interpretation of results	Analysis is reasonable with minor gaps	Superficial analysis; limited interpretation	Unable to analyze or interpret results
Documentation	15%	Presents results clearly with proper documentation and structured reporting	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation